

ENEL300

DESIGN REPORT

PID Flight Platform

Prepared by :

Brendan Booth

Chanelle Duffy

Edward Birchler-Stockdill

CONTENTS

1	Introduction.....	4
2	Context and Requirements.....	4
2.1	Context.....	4
2.2	Primary Stakeholder	5
2.3	Purpose	5
2.4	Design Context	5
2.5	Stakeholder-level Requirements	6
3	Design.....	8
3.1	High level Design Overview	8
3.2	Key Design Decisions.....	9
3.2.1	Power supply testing.....	12
3.3	Mechanical Design	12
3.4	Electrical Design	13
3.4.1	BLDC motors and ESC	13
3.4.2	Power subsystem	14
3.4.3	Control subsystem	14
3.5	Software Design	15
3.5.1	Main (300Project1.ino).....	15
3.5.2	User Interface.....	15
3.5.3	Motors.....	16
3.5.4	Control.....	17
3.5.5	Button	17
3.5.6	Sensor.....	17
3.6	Controller Design	18
3.6.1	Tuning Process	18
3.6.2	Proportional	19
3.6.3	Integral.....	19
3.6.4	Derivative	20
4	Evaluation	21
5	Retrospective and Remaining Tasks.....	22
5.1	Retrospective	22
5.1.1	Team Performance	22
5.1.2	Remaining Tasks.....	22
	Conclusion.....	23

6	References	24
7	Appendices	25
7.1	Schematics	25
7.2	Primary Stakeholder Meetings Summary	26

1 INTRODUCTION

Control systems are a fundamental part of modern engineering, appearing across aerospace, electronics, robotics and power systems. They are an essential tool in any electrical engineer's toolbox. At the University of Canterbury, students are formally introduced to control systems in third year, where the subject is taught largely at a conceptual and theoretical level.

This project seeks to bridge the gap between theory and practice by developing a PID Flight Platform that provides a clear, tangible demonstration of control system dynamics. The aim is to give students an intuitive understanding of how proportional, integral and derivative terms affect system behaviour, complementing the theory taught in ENEL321.

This report outlines the design and development of the Flight Platform. It is structured as follows:

- **Context and Requirements** – background, purpose and stakeholder needs.
- **Design** - a high-level overview followed by detailed electrical, software and mechanical subsystems.
- **Evaluation** - a summary of evaluated results against requirements. The full evaluation can be found in the evaluation report
- **Retrospective** – reflections on project progress, decision-making and team culture
- **Remaining Tasks**- improvements and future work before handover to the primary stakeholder, Chris Hann.
- **Conclusion**- lessons learned and overall outcomes.

By combining practical hardware with user-adjustable PID control constants, the project proves a valuable teaching aid that helps make abstract concepts visible and interactive.

2 CONTEXT AND REQUIREMENTS

2.1 CONTEXT

Control systems are an extremely important topic in electrical engineering. Any truly autonomous technology requires a control system of some kind, as they must sense, decide, and act in response to their environment. In industry, the most used controller is PID (Proportional-Integral-Derivative) control, largely due to its reliability and simplicity [1]. PID control is a core topic of the electrical controls course ENEL321 taught by Christopher Hann, however the subject currently lacks any clear physical demonstration of the concepts.

A control system operates as a continuous feedback loop that takes in sensor information, compares the current state to a desired setpoint, and applies corrective actions based on this difference. A PID controller is a type of control that is made up of three core parts- proportional, integral, and derivative- that each have their own benefits and drawbacks. When used together and tuned by adjusting their respective control constants, a PID control can give a very accurate response. If not tuned correctly, the system can exhibit a range of interesting and often unstable behaviours such as sustained oscillations, slow settling times, or even complete instability [2].

The goal of our project is to design a teaching aid to physically demonstrate these interesting behaviours for our key stakeholder Chris Hann for use when teaching the ENEL321 course.

There are a few other teaching aids developed to demonstrate similar concepts such as the Heli-rig used in embedded systems (ENCE361), the rocket roll rig currently used in ENEL321 and the small desktop single propellor PID system currently used in 321 Lectures.

However, these other projects contain gaps when it comes to the specific topic of simple PID control demonstration. The Heli-rig was developed for ENCE361, and despite using a control system, its main emphasis is on demonstrating the embedded systems with no option of easily adjusting gain values for the control system. This system has also proven unreliable due to its custom-made pieces breaking and being difficult to replace. Whereas the rocket roll rig demonstrates the theoretical side of PID control with the use of graphs but lacks a clear physical demonstration of the effects of control. Leaving students trying to match a reference line on a graph with no idea of what the line represents in a real-world control system. Finally, the small unit currently shown in lectures is prone to the disturbance of the air conditioning dampening the system. Students also struggle to see the effects due to the size of it, missing the intricacies of PID control.

This research highlights a clear gap in the current teaching tools available for the ENEL321 course. There is a need for a device that can physically demonstrate the effects of PID control on a real-life system in a way that is simple and intuitive.

2.2 PRIMARY STAKEHOLDER

The primary stake holder and user for the PID Flight Platform is Chris Hann, the third-year control systems lecturer.

2.3 PURPOSE

While existing demonstrations such as simulations or the ENEL361 helicopter rig offer some visualisation, they lack the real-time physical interaction needed to reinforce student understanding early in the course. Many students struggle to connect simulated data to real-world behaviour. This project aims to develop an engaging, physical flight rig that clearly demonstrates how PID control affects system dynamics, helping students build an intuitive understanding from the outset.

Project mission statement

“To design, develop and test a safe and portable device that physically demonstrates PID control system dynamics, using an engaging flight platform”

2.4 DESIGN CONTEXT

The design will be used to demonstrate how a PID controller can effect system dynamics. This will help future electrical engineers bridge the gap between control systems concepts and real-world tangible control.

A secondary use that has been identified by Chris Hann, is to use the PID hover platform on university open days and electrical department days to entice students to specialise in electrical engineering.

The context of the design is shown in Figure 1.

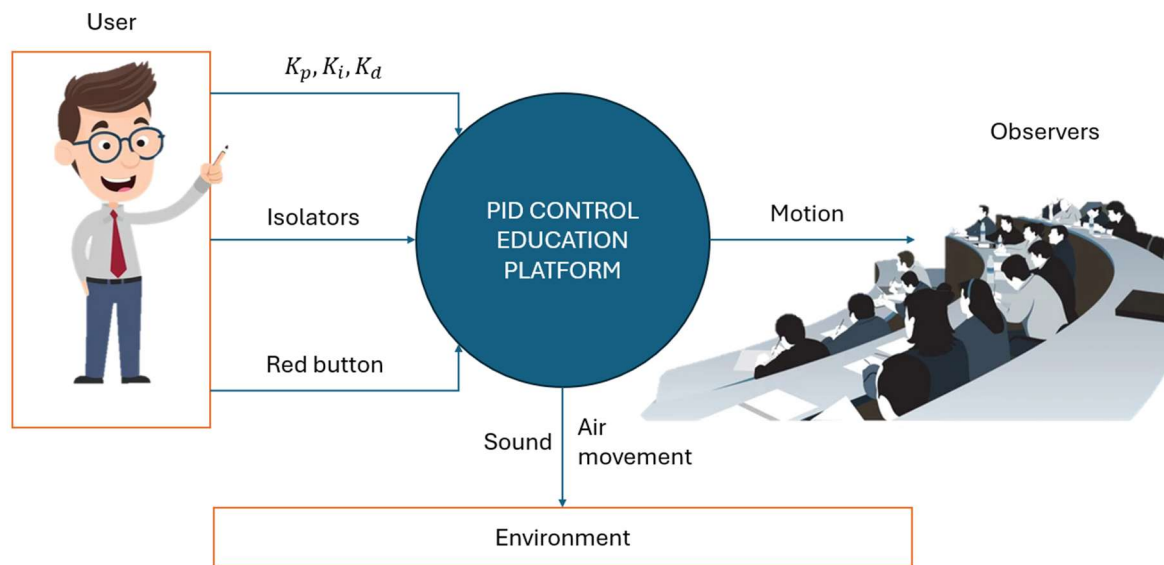


Figure 1 - Design context diagram

As is seen in Figure 1, the design is highly independent, with low external coupling. All operationally necessary components are within the design scope. The user does not require anything other than a way to push buttons, turn knobs, and operate switches. Fingers will suffice.

2.5 STAKEHOLDER-LEVEL REQUIREMENTS

Stakeholder-requirements were established at project inception, and the development of these requirements is covered in the project proposal document. All stakeholder level requirements are listed in Table 1 below. Some latitude is afforded due to the variable priority level of the requirements.

Table 1 - Complete list of stakeholder requirements

No.	Requirement	Acceptance Criteria	Rationale	Priority
1	Total cost of build under \$70	Yes/No	Requirement of project brief (in the absence of stakeholder funding)	Desirable
2	Flight platform is constrained to the z-axis position.	Yes/No	Addition of Pitch, yaw and roll would be too complex to be completed within the timeframe.	Essential
3	Flight platform range of motion greater than 300mm	Yes/No	Students need to be able to see the response of the different controls during a lecture demonstration.	Essential
4	A PID controller is controlling the movement of the flying platform.	Yes/No		Essential

5	The control constants each have three preset values including zero	Yes/No	Provides sufficient combinations to demonstrate how PID tuning affects dynamic response.	Essential
6	Variation in control dynamics between different controller combinations	In a randomised demo of 5 presets, the primary stakeholder correctly identifies relative response differences in $\geq 4/5$ cases.	Students should be able to see the changes in system response for different controller combinations.	Essential
7	Control constants are changed via a dedicated user interface.	Yes/No	Familiarity with program code should not be required to operate device	Desirable
8	Control constants adjustable between flights within 1 minute	Yes/No	Fast change of control variables ensures the students will remain engaged throughout.	Essential
9	The control constants each have six preset values including zero	Yes/No	Though not essential more combinations of constants can help with demonstrating effects of PID aiding in student understanding.	Optional
10	Flight platform sustains flight for at least 10 seconds	Yes/No	The minimum time required to demonstrate the effect of control dynamics	Essential
11	The flight platform is enclosed to prevent contact with any moving or electrically active components.	Yes/No	To prevent contact with potentially harmful components.	Essential
12	Conforms to all University safety regulations.	Yes/No		Essential
13	Flight platform enclosure does not hinder visual observation.	The material used for the enclosure is transparent.	Students need to be able to see the response of the different controls during a lecture demonstration.	Essential
14	Transportable to and from lecture theatre	Yes/No	To have the option of a lecture showcase.	Desirable
15	Flight platform responds to external disturbance applied by user.	Yes/No	To have the option of demonstrating the effect of disturbances.	Optional
16	Ground effect, turbulence and frame should not	Yes/No	The ideal PID system response should be clear;	Desirable

	cause the flight platform's position to deviate by more than ± 100 mm from the settled point.		these sorts of disturbances will affect the clarity of the response.	
17	Control constants can be changed while the platform is in flight.	Yes/No	Allows for more efficient demonstrations as the platform is not required to land each time the parameters are changed.	Optional

3 DESIGN

3.1 HIGH LEVEL DESIGN OVERVIEW

A high-level functional block diagram of the prototype design is shown in Figure 2 below.

The primary physical components of the prototype are a flight platform with BLDC motors and props mounted on it, constrained within cuboid guide frame with four vertical rods, a separate control panel, and a battery.

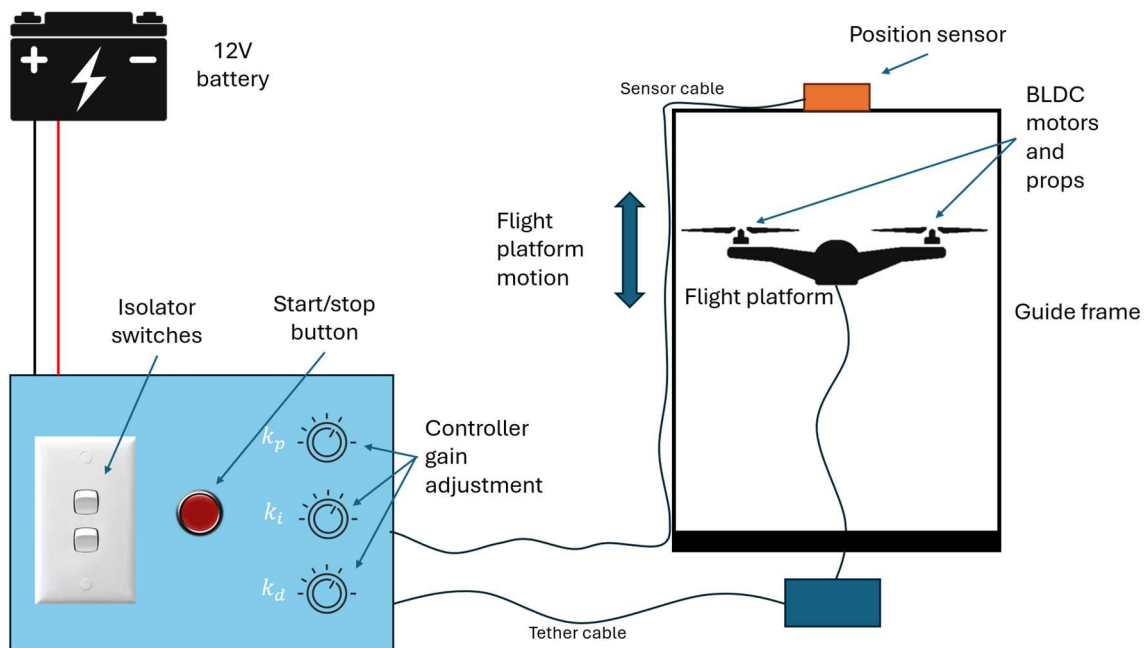


Figure 2 - High level block diagram

The flight platform motion is used to demonstrate control system dynamics. The platform is free to move vertically and is levitated by four BLDC motors and props. By using a PID controller to control the motor speeds to try and maintain a set height, the control system behaviour is translated into the vertical motion and position of the platform, as shown in Figure 3.

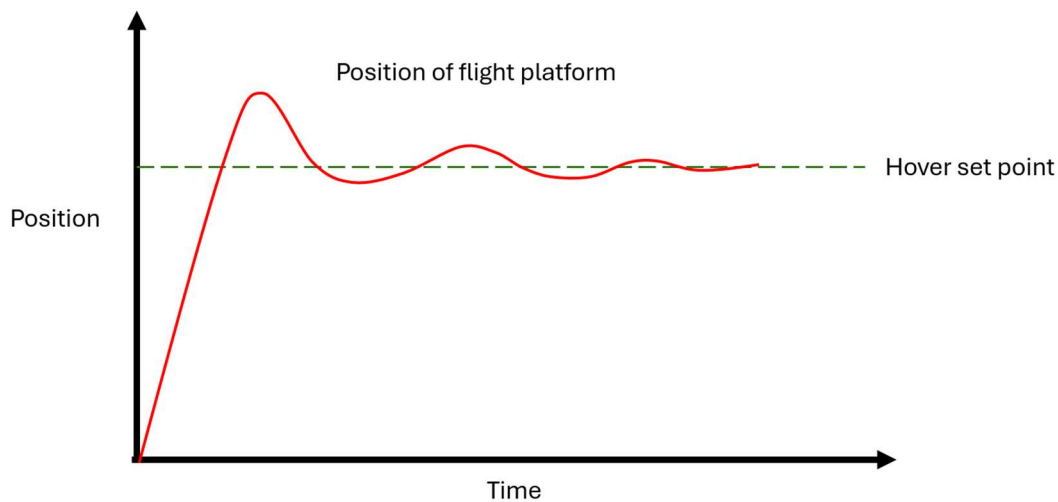


Figure 3 - Flight platform position demonstrating PID control

A VL53L1XV2 LiDAR time-of-flight (ToF) sensor is mounted on the top of the frame and measures the distance to the flight platform. The Arduino Uno microcontroller in the control panel controls the motors via a tether cable to the flight platform. Mounted on the control panel are three potentiometers to adjust the control system gain constants, a momentary push button to start and stop the motors, and two isolator switches for the motor speed controllers. A separate 12V SLA battery provides power via a flexible cable.

Breakdown of the complete mechanical, electrical, and control systems are provided in detail in sections 3.3, 3.4, and 3.5.

3.2 KEY DESIGN DECISIONS

Considering the proposed design as a whole, key decisions were made to ensure that the design specifications would lead to a prototype that would meet the stakeholders' requirements. Considerable research and investigation were undertaken to determine the feasibility of various elements of the design.

Listed below, in chronological order, are the key design decisions, with justification, made over the course of design development.

1. Motor arrangement

A four-rotor quadcopter design was selected to improve flight platform stability and to allow for external disturbance to be applied in the form of force applied directly to the flight platform at the centre of mass. This will provision for the potential extension requirement 15.

2. Motor selection

The motors must be capable of generating sufficient lift. Brushed DC motors were initially considered as these are used in some hobby RC aircraft applications, are budget friendly, and have simple speed control attributes. However, BLDC motors are near ubiquitous in UAV/drone applications and provide key advantages such as high power to weight ratio, high efficiency, and back EMF for positional control. BLDC motors cannot be powered

directly with DC and often use electronic speed controllers (see Figure 4, Section 3.4.1). The alternative is to use a microcontroller PWM signal to provide the required rotating signal to the motor windings. Table 2 below was used to evaluate motor options, and the decision was made to use ESC + BLDC motors. The final prototype uses A2212 BLDC motors and 30A ESCs, selected for availability and budget reasons.

Table 2 - Motor selection s

Parameters	ESC + BLDC	BLDC	Brushed
Software complexity	Low	High	Low
Hardware complexity	Low	High	Low
Startup behaviour	Good (Smooth)	Bad (Requires extra control)	Good
Low-speed control	Can be unstable (depends on ESC quality)	Unstable (choppy)	Good
High-speed control	Good (Smooth)	Good (Smooth)	Medium (brushes get hot)
Wiring	3 Wires	3 Wires	2 wires (+/-)
Maintenance	Low	Low	High
Reliability	Very Reliable	Depends on design	Reliable (simple)
Safety	Built in protection	Must be implemented	Need flyback diodes
Amount	Need at least 2 ESCs per 4 motors	4	4
Availability	Expensive in NZ, cheaper overseas	Expensive in NZ	Available
Responsivity	Very good	Good	Poor
Weight to lift ratio	Very good	Very good	Poor
Cost (unit price)	\$50+	\$20 +	\$10

3. Tethered vs. non-tethered flight platform

The flight platform shares many functional similarities with hobbyist grade drones, which are powered with onboard Li-po batteries. Given the limited range of motion for this design, and the cost barrier of Li-po batteries, as well as the requirement for control, a tethered design was chosen.

4. Power supply

BLDC motors required to provide lift can draw up to 50A at 12V. This exceeds the available supply from most benchtop DC supplies. Given the portability requirements and budget, a 12V SLA battery was selected. More detail on the testing performed to support this decision is given in section 4.2.1

5. Digital PID control.

Given the requirement that the controller gains are to be adjustable by the user, implementing this with analogue PID control would be very complex.

6. Sensor type

Given the time constraints of this project, it was decided that a distance sensor would be used, as opposed to something more complex such as an IMU. There were four key considerations to take into account when searching for a distance sensor: cost, availability, sampling frequency, and range of accuracy. A comparison table of these considerations can be seen in Table 3 below. Note that the data in this table is based upon sensors that could be obtained within the time frame [3] [4] [5].

Table 3: Comparison of different distance sensor types against system requirements.

Considerations	IR	ToF	Ultrasonic
Max Sampling Frequency ($f_{s,max}$)	200 Hz	50 Hz	16.67 Hz (Recommended)
Accurate range at $f_{s,max}$	~ 20 – 150 cm	4 - 130 cm	2 – 400 cm
Availability	Available	Available	Available
Cost	Low	Medium	Low

For a responsive control system, we required a sensor that could sample at around 50Hz. This was deemed sufficiently high enough given the latency introduced through the ESC's and motors. On top of this, the range of measurement had to be accurate from as close as 10mm up to 500mm. As seen in the table above, the ToF sensor matched the closest to what was required. IR had a high sampling rate but a very high minimum range, whereas the ultrasonic has an ideal range but a very low sampling frequency.

The ToF sensor was chosen for this design project given the requirements and constraints.

7. Microchip/Development Board

Due to the time constraints, it was decided that a development board would be used for this project. The board chosen had to meet the following conditions in order to be useable for our purpose.

The development board required:

- 2x PWM pins
- 3x Analogue ADC pins
- 3x Digital pins
- I2C communication lines (SCL/SDA)

Optional extras:

- A built-in voltage regulator that can handle up to 15V input (the board can be powered from the same supply as the ESC's/motors).
- Internal pull-up/pull-down resistors for GPIO pins.
- 3.3V I2C communication lines (The ToF sensor runs on 3.3V).
- A 3.3V output voltage.

Based on these requirements, the Arduino Uno R3 was chosen. This board meets all of the requirements laid out above. It consists of 14 digital pins, 6 analogue pins, 6 PWM pins, two

sets of I2C communication lines, and a barrel jack that runs nominally in the 9-12V range but can safely handle up to 20V input. On top of that, it has both a 5V and a 3.3V output pin and certain digital pins have built in pull-up resistors.

The Arduino Uno R3 I2C communication lines run on 5V, this is too high to connect directly to the ToF sensor mentioned above. Therefore, the use of a bi-directional logic level converter was necessary to communicate between Arduino and sensor.

3.2.1 Power supply testing

The datasheet for the selected A2212 BLDC motors states maximum power of 150W and maximum current of 13A for periods of up to 60 seconds [6]. Testing of individual motors showed highly changeable current draw, from 0.6A at no load, to peak inrush currents when accelerating from standstill to max speed approaching 20A, more than the maximum current stated on the datasheet. Motor acceleration was slower when the current was limited to the 10A available from laboratory DC supplies, and the requirements for dynamic and responsive speed control for the control system specified a power supply can meet the inrush current requirements for all four motors.

With an appropriate margin of error, allowing 20A for each motor, a supply capable of delivering 60A peak is required. This is not an application suited for benchtop DC power supplies. Informed by the standard use of Li-Po batteries in drones, a battery was determined to be the most suitable supply.

Testing was carried out with different batteries available for use at no cost. Three 1.2Ah 12V SLA batteries in parallel proved to deplete too rapidly for extended operation. Further testing on an early revision of the prototype indicated that the motors require approximately 4A each to maintain a hover. A higher capacity 650CCA 12V SLA automotive starting battery was substituted, and this proved suitable. Calculations based on the average hover current of 4A per motor, and the battery amp-hour rating of 65Ah, reserve capacity rating of 110 minutes, suggest that this battery will be capable of continuously powering the prototype for over an hour. Detail on this testing procedure and calculations are provided in the evaluation report.

3.3 MECHANICAL DESIGN

The design of the mechanical section can be broken down to the flight platform, the base and the surrounding acrylic enclosure. The flight platform was 3D printed PLA. This is comprised of four arms, each with one motor. These arms are connected to a central octagon via dove tail joints. The system initially was built around being very robust coming at the cost of weight. The design was later refined into a much more lightweight platform to assist the propellers with a faster response, helping to demonstrate system dynamics.

The base is made up of four 8mm stainless steel rods glued into a 495*495 piece of MDF. This proved rigid enough to keep the flight platform constrained to the Z-axis. The base was placed on 100mm high 3D printed stand legs to allow the flight cable to move freely. Finally, the display case is 3mm laser cut acrylic to comply with safety requirements at the University of Canterbury.

3.4 ELECTRICAL DESIGN

The overall electrical design is built around the specifications of key components, namely the A2212 BLDC motors and 30A ESC units, the VL53L1XV2 sensor unit, and the Arduino Uno microcontroller. The design is subdivided into the power related subsystem and the control related subsystem. The power system concerns the supply, isolation, and delivery of electrical power to the microcontroller and the BLDC motors via the power stage of the ESCs. The control system concerns the sensor inputs and user inputs to the microcontroller, and the signal outputs to the ESCs to control the BLDC motors. The complete schematic is included in Appendix 7.1

Much of the electrical hardware is mounted in the control panel, including the Arduino Uno, the ESCs, the user controls, and most electrical connections. Only the battery, motors, and sensor are external by necessity. For this prototype, the control panel was built from a Sistema lunch box container. This decision was made to minimise impact on budget and because it allowed user input devices to be mounted easily.

3.4.1 BLDC motors and ESC

BLDC motors require an ESC to operate. The 30A ESC units paired with the A2212 BLDC motors takes as an input a constant 12V source for power, and a 5V 50Hz PWM signal with duty ratio varying from 5-10% to control the motor speed. The inbuilt microcontroller switches the three motor windings as required to deliver the requested speed. This simplifies the motor control a great deal. A simplified representation of this operation is shown in Figure 4 below.

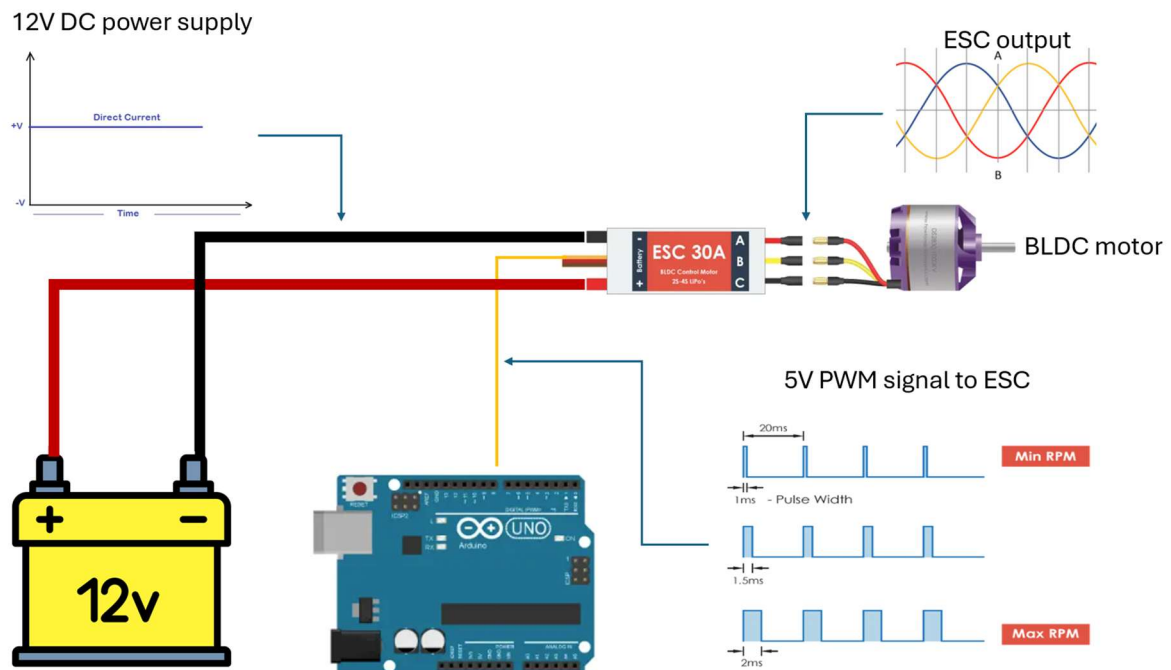


Figure 4 - BLDC motor and ESC operation

Individual motor control is not required. Therefore, each motor does not require a dedicated ESC. Two motors are operated off each ESC, with one ESC driving the diagonally opposite motors clockwise, the other ESC driving the remaining two motors anticlockwise. Benchtop testing was performed to ensure that one ESC would reliably synchronize and drive two BLDC motors.

3.4.2 Power subsystem

Each of the four BLDC motors can draw a maximum current of 13A when delivering peak power [6]. This current draw was independently confirmed during testing using the laboratory DC power supplies. Furthermore, current requirements fluctuate significantly during operation with peak inrush currents significantly more than 13A. Responsiveness and acceleration of the motors is compromised when the supply cannot respond quickly.

This requires a supply capable of providing more than 60A rapidly and without excessive voltage drop, but only for short periods. Batteries are well suited to this rapid, high current behaviour. Available for use at no cost was a 650CCA 12V SLA automotive starting battery. This battery is rated to deliver 650A for 30 seconds at 0°C, which is more than sufficient for this application.

The battery is connected to the 12V bus which supplies all 12V devices. The Arduino Uno is supplied via a barrel jack connector with 12V, and the internal voltage regulator is utilised to provide the 5V supply for the microcontroller. 12V is the stated upper limit for powering the Arduino [7], and caution is advised as the voltage regulator may heat up when dissipating ~7V under higher load conditions. However, the input power to the Arduino is minimised by using external supplies, and the total input current was calculated at ~100mA, well below the input current limit of 700mA.

The ESCs are mounted in the control panel. Each ESC is supplied through a 32A rated isolator switch. A 2m length of 6 core 0.75mm² control cable runs from the control panel up through the base of the guide frame to the flight platform and is terminated at the motor terminals. Three cores carry the clockwise signal to diagonally opposite motors, and the remaining three cores carry the anticlockwise signal to the other two motors. This cable is subject to movement and stress and is appropriately rated as per AS/NZS 3191:2008 and glanded for security.

3.4.3 Control subsystem

Motor Control

The ESC units contain a built-in microcontroller which interprets a 40Hz PWM control signal and controls the power outputs to the motor to deliver the required speed. The ESC units also contain a battery eliminator circuit, meaning the inbuilt microcontroller does not require a dedicated supply separate from the 12V power supply [8]. A 5V 40Hz PWM signal with pulse width of 1-2ms from the microcontroller to each ESC is all that is required.

Sensor input

The sensor signal is required to inform the microcontroller of the instantaneous position of the flight platform. Initial design used an ultrasonic sensor, but as the design progressed the limited frequency of the ultrasonic sensor rendered it unsuitable. The VL53L1XV2 LiDAR sensor was substituted as it operates at a higher frequency and is accurate across the distance required. However, due to the sensors 3.3V I2C lines a 3.3V to 5V logic convertor was necessary to communicate between the sensor and Arduino. The sensor is connected to the microcontroller via the logic level converter with a 2m length of 6-core signal cable.

User input

The user inputs are mounted on the control panel. Three 0-10KΩ linear potentiometers connected to three ADC pins on the Arduino allow adjustment of the analogue voltage to these pins, which changes the control system gains. A single red momentary push button connected to digital GPIO pin 3 with an internal pull-up resistor enabled is used to start and stop the motors.

3.5 SOFTWARE DESIGN

The software design was developed with a bottom-up approach. Each individual module was made separately with the main file connecting them at the end. Due to the nature of our design having very distinct parts, the software was able to be designed modularly as seen in Figure 5 below. All modules use the <Arduino.h> library and are written in Arduino IDE, the full code can be found in the git repository [9]. The following section will outline the core modules and their purpose.

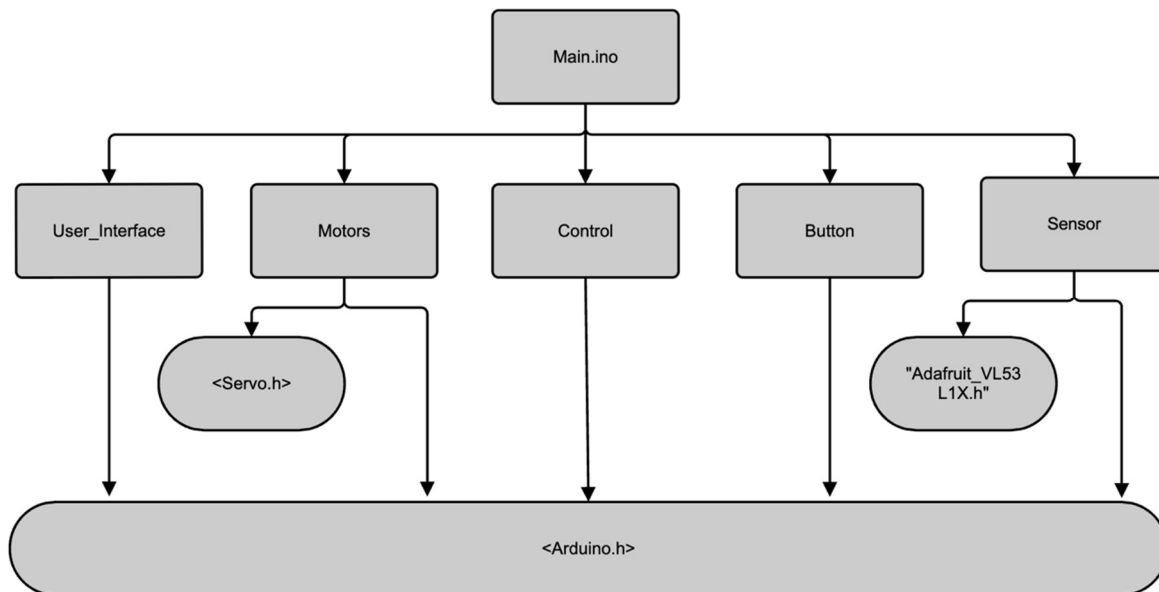


Figure 5: Software Block Diagram

3.5.1 Main (300Project1.ino)

The main file consists of two core functions.

- `setup()`, this handles the initialisation of all modules, it calls the setup functions of the other cpp modules.
- `loop()`, this is the main loop that handles timing and the passing and pulling of variables. Due to the sensor sampling limit of 40Hz, this system runs every 25ms calling the various functions outlined below.

The main file handles the linkage between the .cpp files, the exact variables passed are defined in the coming sections. The full code can be found at:

<https://eng-git.canterbury.ac.nz/eab109/enel300-project-1.git>

3.5.2 User Interface

The user interface module handles the updating and mapping of the potentiometer values to gain values. There are three distinct gain options able to be chosen through the potentiometer user interface, this module reads these ADC values and outputs the chosen gain values. The main functions are as follows.

- `pots_setup()` called in main setup, handles the initialisation of the analog pin inputs.

- `update_pots()` called in main loop, reads the current potentiometer values and maps them to the relevant gain values through helper functions.

Main pulls the gain values from this module through proportional, integral and derivative getter functions.

3.5.3 Motors

The motor module handles the driving of the motors through the `<Servo.h>` library. It is passed the PWM value produced from the control loop, and it drives the motors based on this. On top of this it also handles the landing procedure and the arming of the ESC's.

- `motor_setup()` is called in the main setup function; this handles the arming and initial setup of the ESC's.
- `motor_update(PWM signal)` this function takes inputted PWM signal produced from the control module and drives the motors with this value. The motors will only be driven if there is power supplied to both ESC's, this is a safety feature to stop all motor movement in the case of any loss of power. A snippet of this function can be seen in Code snippet 1 below, this shows the cutoff and the re-arming sequence required for the ESC's after power loss.
- `Landing_procedure()` this is a simple function that gradually reduces the motor speed to land gently.

```
void motor_update(int escSignal) {

    bool esc1Power = digitalRead(ESC_POWER_PIN_1) == HIGH;
    bool esc2Power = digitalRead(ESC_POWER_PIN_2) == HIGH;

    if (esc1Power && esc2Power && !escArmed) {
        // Both ESCs just powered back on, re-run arming
        armESCs();
        escArmed = true;
    }
    else if (!esc1Power || !esc2Power) {
        // Even if one ESC lost power, stop both
        escArmed = false;
        ESC1.writeMicroseconds(ESC_MIN_PULSE);
        ESC2.writeMicroseconds(ESC_MIN_PULSE);
    }

    // Send normal signal if both armed
    if (escArmed) {
        ESC1.writeMicroseconds(escSignal);
        ESC2.writeMicroseconds(escSignal);
    }
}
```

Code Snippet 1: Motor update function.

Main passes the PWM value into the `motor_update` function which communicates with the ESC's and drives the motors.

3.5.4 Control

This module contains two functions, the initialisation function and the PID control function. The measurement from the sensor is passed into this module and a PWM offset is outputted. The control module uses a struct called PIDController that contains the gain values, the sampling time, the controller limits, the PID control outputs and the previous sensor measurement.

- PIDController_init(PIDController *pid, float setpoint, int16_t measurement) initialises the control variables on startup. This is called in the Main setup() function.
- PIDController_Update(PIDController *pid, float setpoint, int16_t measurement), this function takes in the current sensor reading and uses it to calculate the proportional, derivative and integral components. The output is the summation of these components.

The original code for the PIDController_Update function was pulled from an open source git [10]. This code served as the basis to the controller; it was then edited and adjusted to fit our purpose.

3.5.5 Button

The button module handles the logic for the on/off button. It reads the state of the digital pin 3 on the Arduino and outputs a button state Boolean that is called through a getter function in main.

- Button_setup() initialises the pinmode through the <Arduino.h> library.
- Button_update() reads the pin value and handles the logic for the button press.

3.5.6 Sensor

The sensor module handles the reading, converting and filtering of the sensor data. It utilises the Adafruit library specific to the type of sensor being used (VL53L1X). This library handles any conversions needed and produces a distance in millimetres through the .distance() function. There are four main functions in this module, two for filtering (median of three and EMA) plus the core setup and update functions.

- Sensor_setup(), this module sets the sensor sampling rate and ensures the sensor communications are working as expected. Error handling is in place if the sensor cannot be found.
- Filter_update(new sample), this function handles the updating of a small circular buffer that is fed into the median of three filter seen in Code Snippet 2. The return value of this filter is passed through an exponential moving average filter with an $\alpha = 0.4$ value (Equation 1 below). These filters combined produced a smoothed sensor output, essential for the derivative control component.
- Sensor_reading(), this function reads the new sample and passes it into the filtering function above. This function is called in the main loop() function and it returns the final filtered measurement value.

```

int16_t median3(int16_t a, int16_t b, int16_t c) {
    if ((a >= b && a <= c) || (a >= c && a <= b)) return a;
    if ((b >= a && b <= c) || (b >= c && b <= a)) return b;
    return c;
}

```

Code Snippet 2: Median of Three filtering helper function.

$$\text{filtered output} = \alpha * \text{median3 output} + (1 - \alpha) * \text{prev filtered output.} \quad \text{Equation 1}$$

3.6 CONTROLLER DESIGN

To design a viable PID controller, an understanding of how it incorporates into the rest of the system is required. The block diagram found in Figure 6 below outlines inputs and outputs of the PID control block. This controller runs on height feedback in millimetres, fed from the ToF sensor. Due to the variation in sensor data this measurement is fed through noise filtering on the feedback line, before being acted on by the control system.

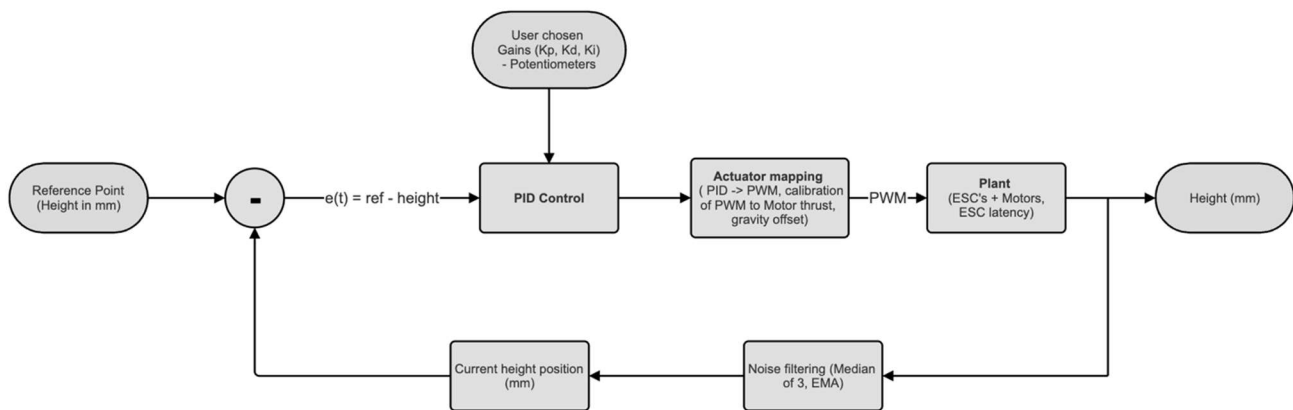


Figure 6 - Control diagram for the flight platform system

The purpose of our design is to demonstrate the effects of different PID and gain combinations on a physical system. As outlined in our requirements, it was decided that there would be three distinct gain settings for each K_p , K_i , and K_d . In order to clearly demonstrate the different tuning, the set gain values were chosen to give an exaggerated response. The process of calculating and tuning the PID controller is outlined below.

3.6.1 Tuning Process

Due to time constraints, the tuning process was primarily trial and error. Given more time, the tuning would have been approached mathematically using system ID and the methods taught in ENEL321 such as root locus. Despite this, the tuning was approached with a systematic process.

- 1) Gravity offset:** To simplify the controller mathematics, a gravity offset was necessary. This ensured the PID control was only handling corrections around the setpoint. Through the gradual increase in PWM, an offset was found that allowed the flight platform to hover at the desired setpoint, in our system this offset was a duty cycle of 43.5%.

- 2) **Controller Bounds:** From this gravity offset, a minimum and maximum output bound was found for the controller. These bounds clamped the PWM output within a safe operational range. These had to be chosen carefully as our system has to safely operate proportional and integral controllers without the derivative damping component. Through trial and error, it was found that the maximum safe duty cycle was the gravity offset +200, similarly the minimum duty cycle was found to be the gravity offset -200.
- 3) **Gain Tuning:** After these bounds were determined, the rest of the tuning relied on the gain values. The proportional controller was tuned first as it was the most intuitive, followed by the integral control which had the extra element of anti-integral-windup. Finally, the derivative controller was tuned. A more in-depth explanation of each can be found below.

3.6.2 Proportional

The proportional component is calculated using Equation 2 below, where the error is the difference between the current position from the sensor reading and the setpoint.

$$proportional = K_p * error \quad \text{Equation 2}$$

The proportional component was tuned to give the desired response by starting with low K_p values and slowly increasing. In the case of our project, this controller had to demonstrate the typical overshoot caused by the proportional element, followed by large oscillations that gradually reduce to small movements around the setpoint.

A K_p value of 0.18 was found to give the sought after response, this was set as the high K_p that can be chosen by turning the proportional potentiometer all the way to the right.

The medium proportional value was determined after the tuning of the integral controller, as it was used to represent the response of combined controllers such as PI and PD. From experiments it was found that a K_p of 0.15 worked well with both the integral and derivative components.

The low value for proportional control was zero, as this allowed for the demonstration of an integral controller.

3.6.3 Integral

The integral component was calculated using Equation 3, where T is the time step which is 25ms (40Hz).

$$integral = prev\ integral + K_i * T * (error + prev\ error) \quad \text{Equation 3}$$

The integral component was first tuned individually ($K_p = 0$) to get the desired response. For a purely integral controller, the behaviour of the system should show the build-up of error at the start until it accumulates enough to take off from the ground. Due to the accumulated error the flight platform should overshoot, then correct itself by building up negative error causing it to drop below the setpoint, before eventually settling with no further oscillations.

The integral controller was tuned through the same method as before, starting small and slowly stepping up the K_i value until the system reflected the behaviour expected. However, unlike the

proportional controller, extra bounds had to be implemented on the integral control to prevent large wind-up. To tune the integral control, both these bounds and the K_i values were experimentally changed to determine what combination worked best.

Due to the unique situation of wanting to demonstrate both a pure integral controller and a combination of controllers, multiple methods of anti-windup were implemented. The first being simple maximum and minimum bounds. This method was used for just the integral controller, as it allowed for the demonstration of the wind-up at the beginning when it is building up error.

When demonstrating a combination of controllers such as PI or PID, the slow wind-up at the start is unwanted as it interferes with the ideal behaviour of these other types of controllers. To overcome this, if there is a proportional component ($K_p \neq 0$) the integral component won't do anything until the error is less than 30mm.

For this system, the integral limits were set to be +50 and -50, and the high K_i value was set to be 0.02. This is to demonstrate the effect of an integral controller. The medium K_i value was set to 0.01, as this value worked well with the K_p value to demonstrate a PI controller.

As before the low K_i value is zero.

3.6.4 Derivative

The derivative component calculation (Equation 4) is slightly more complicated than the previous two as it includes a low-pass filter. The derivative component will naturally amplify any noise from the sensor due to the difference calculation between the previous measurement and the current measurement. The ToF sensor was found to still have a variation of $\pm 5\text{mm}$ after filtering, therefore the low-pass filter was set with $\tau = 0.4$. This reduced the effect of the sensor variation. As defined above, the time step $T = 25\text{ms}$.

$$\text{derivative} = (-2 * K_d * (\text{prev measurement} - \text{measurement})) + \frac{2\tau - T}{2\tau + T} * \text{prev derivative}$$

Equation 4

As the derivative component acts only on velocity, a derivative controller is not going to do anything on its own (excluding the effect of the gravity offset). Therefore, the tuning around this derivative component was conducted with a proportional component. The ideal response of this PD controller is to demonstrate damping, the medium K_d value was setup up to demonstrate slight damping whereas the high K_d was to show an overdamped system.

The tuning of this PD controller was conducted by setting the proportional gain value and experimenting with the effect of the derivative component on this. As defined above, the medium K_p value has already been set up to work with the integral component. Therefore this K_p value was used as a setpoint that the derivative controller could be tuned around.

Through similar method of starting low and building up, a medium and high K_d of 0.05 and 0.08 were found.

As before the low value of K_d is zero.

4 EVALUATION

Of the requirements identified at the proposal stage, requirements 1, 6, 9, 17 were not met. These requirements and the rationale are reproduced in Table 4.

Table 4 - Unmet requirements

No.	Requirement	Acceptance Criteria	Rationale	Priority
1	Total cost of build under \$70	Yes/No	Requirement of project brief (in the absence of stakeholder funding)	Desirable
6	Variation in control dynamics between different controller combinations	In a randomised demo of 5 presets, the primary stakeholder correctly identifies relative response differences in $\geq 4/5$ cases.	Students should be able to see the changes in system response for different controller combinations.	Essential
9	The control constants each have six preset values including zero	Yes/No	Though not essential more combinations of constants can help with demonstrating effects of PID aiding in student understanding.	Optional
17	Control constants can be changed while the platform is in flight.	Yes/No	Allows for more efficient demonstrations as the platform is not required to land each time the parameters are changed.	Optional

It was anticipated that as the project progressed changes and modifications to the initially identified requirements might be required. In the case of requirement 1, stakeholder funding was secured above the baseline project budget, superseding this requirement. Requirements 9 and 17 are extensions of more fundamental functional requirements, and although not met at this time by the prototype, the design provides for simple integration during subsequent development.

Requirement 6 was primarily due to the strict criteria for success defined at proposal stage. At the demonstration the primary stakeholder identified correctly three control responses, because only three tuned controllers were prepared for the demonstration due to time constraints. The design can meet this requirement with minimal work.

A more complete explanation is provided in the evaluation report.

5 RETROSPECTIVE AND REMAINING TASKS

5.1 RETROSPECTIVE

Our approach to the decision-making process outlined in the proposal where each member is responsible for one key area (Electrical, Software, Mechanical) worked incredibly well. This allowed for highly independent design development in the initial stages, with each member taking responsibility for their section and progressing those aspects of the design independently.

Bringing these areas together required good communication in the middle to later stages of the project about what was required for cohesion. It was anticipated that bringing these separate workstreams together for final implementation would require flexibility, and rapid turnaround with path-critical tasks. This was achieved for the most part, although there were some unanticipated limitations and delays listed below:

- Very limited selection and shipping time of motors. (**Limitation and Delay**).
- Guide rods initially had linear bearings proved to pinch the platform (**Delay**)
- Power supply- Changing from smaller to the larger battery required retuning of controller (**Delay**)
- Elevating the rig off the ground and removing a lot of the base board to allow cable to move freely, meant retuning of controller. (**Delay**)
- Having the acrylic enclosure surround the drone meant retuning of controller. (**Delay**)

The extend of the software controller development tuning and flow-through effects of minor changes to physical system were not explicitly identified at the proposal stage, due to the limited experience of the team with real-world control system implementation. This meant that controller tuning, and development required considerable time and experimentation.

5.1.1 Team Performance

Separation of design responsibility minimised design decision delay for development within designated areas, as team members had autonomy to process without unnecessary consultation. While this approach was successful at accelerating development during the initial stages of the project, some important lessons and limitations of this approach were observed:

- Potential for mismatched workloads.
- No implicit peer-review, design critique, and independent technical assessment.
- Independent work on one aspect (e.g., mechanical changes like the acrylic case) can unintentionally shift workload to others (e.g., software retuning).

In totality, the team is satisfied with overall performance. In future, a distinct shift from independent workstreams to a collaborative development at an appropriate midpoint in the project shall be employed. A structured system for peer review will also be explored to overcome the limitations defined above.

5.1.2 Remaining Tasks

The project is currently at a presentable stage and has already been demonstrated to first-year engineering students who are still exploring their discipline choices. While the demonstration was effective, a few minor improvements are required before handover:

- Replace the prototype Sistema container with a proper electrical junction box to house the electronics.
- Design and 3D print a protective housing for the LiDAR sensor.
- Refine the nine preset values for the control constants to achieve a more responsive system.

However, Chris has stated he would love to see this project developed much further.

- The current LiDAR sensor is slow for sampling, and a better sensor could aid in tuning of presets
- Integration of a live tracking graphical feed to aid in linking physical phenomena with concepts taught in ENEL321
- Currently a logic level shifter is used between the 3.3V sensor and the 5V Arduino Uno. Changing to a microprocessor that has 3.3V I2C communications would minimise circuitry and improve reliability.
- Replace the flight platform tether cable with a silicon sheathed control cable to minimise any effect to system dynamics.
- The integration of an IMU for better control especially around pitch and roll.
- Moving from the hover platform being constrained to the z-axis, to having 6 degrees of freedom.

CONCLUSION

This project set out to design and construct a PID Flight platform that bridges the gap between theoretical concepts of control systems and their real-world behaviour. The flight platform developed, successfully demonstrates how proportional, integral and derivative control terms influence system dynamics. Having already been used in teaching demonstrations with first year students, it validates its effectiveness as an educational tool.

The device met all but one of the essential stakeholder requirements, with the device proving to be safe, and capable of accurately demonstrating system dynamics. The only essential requirement not fully achieved “Variation in control dynamics between different controller combinations” was only partially satisfied, with three presets identified correctly. Further tuning of these presets is needed to more accurately demonstrate the intended dynamics.

Optional/ Desirable Requirements 1, 9, and 17 remain outstanding. These will be addressed in the next stage of development, which will include integrating an IMU for pitch and roll management as well as adding a graphical feed to better link physical behaviour with the concepts taught in ENEL321.

Overall, the project achieved its mission of producing a safe and functional teaching platform that complements ENEL321 by making abstract control theory tangible and interactive. With further refinement the flight platform has potential to become a long-term teaching aid and showcase for the Electrical department.

6 REFERENCES

- [1] C. Bergquist, “How to Build a Drone | A DIY Guide,” *Dojo for Drones*, Mar. 23, 2019. [Online]. Available: <https://dojofordrones.com/build-a-drone/>
- [2] C. Hann, “Lecture C2. Recognise PID control,” *ENEL321 Course notes – Control systems*, pp. 64–65, University of Canterbury, Christchurch, 2025.
- [3] “Ultra-Sonic Ranging Module HC-SR04 Datasheet,” www.Electronics.com.
<https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf> (accessed Sep. 20, 2025).
- [4] “Duinotech Arduino Compatible IR Distance Sensor | Jaycar New Zealand,” Jaycar New Zealand PROD. https://www.jaycar.co.nz/duinotech-arduino-compatible-ir-distance-sensor/p/XC3735?srsId=AfmBOopO9hL-1AmclqmO4WKDZmrF0MABUluYoU7Ww96EqjQQg154V0v_d_c
- [5] “A new generation, long distance ranging Time-of-Flight sensor based on ST’s FlightSense technology,” STMicroelectronics.
<https://www.st.com/resource/en/datasheet/vl53l1x.pdf> (accessed Sep. 20, 2025).
- [6] Rhydo Technologies Pvt. Ltd., “A2212/13T Technical Data,” n.d.
- [7] Arduino, “Arduino – Hardware – Uno R3,” *Arduino.cc*. [Online]. Available: <https://docs.arduino.cc/hardware/uno-rev3/>
- [8] Optimus Digital, “30A BLDC ESC datasheet,” n.d.
- [9] “eab109 / ENEL300 Project 1 · GitLab,” GitLab. <https://eng-git.canterbury.ac.nz/eab109/enel300-project-1>
- [10] Pms, “GitHub - pms67/PID: PID controller implementation written in C.,” GitHub. <https://github.com/pms67/PID>

7 APPENDICES

7.1 SCHEMATICS

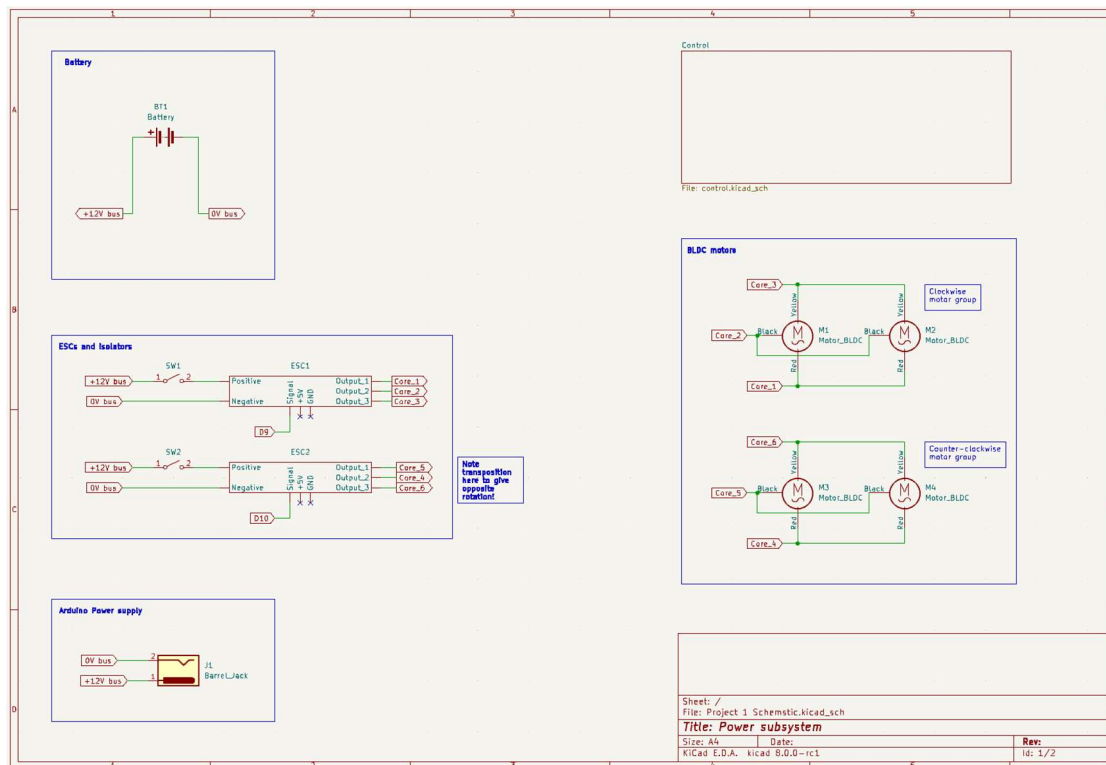


Figure 7 - Power subsystem

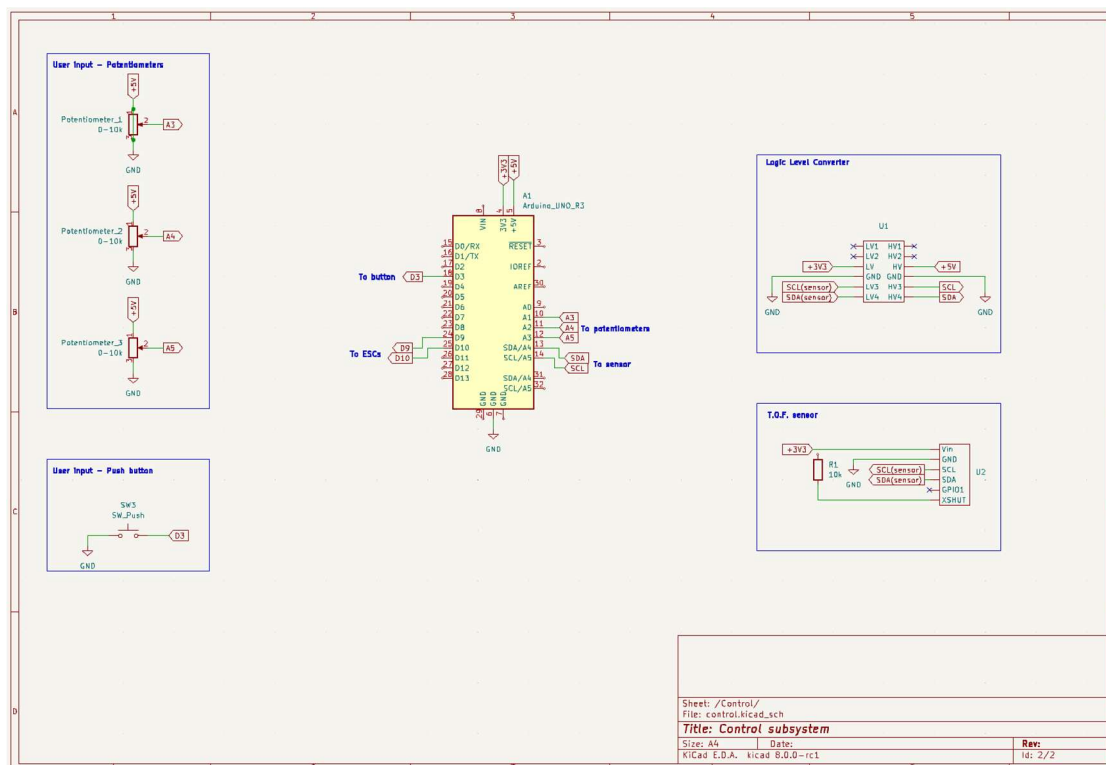


Figure 8 - Control subsystem

7.2 PRIMARY STAKEHOLDER MEETINGS SUMMARY

Compact summary minutes of main stakeholder meetings are provided below.

1) Project expression of interest/pitch

Attendees	Chris Hann, Brendan Booth, Chanelle Duffy
Date	17 th August
Location	Link Rm 502
Discussion points	Availability to be primary stakeholder. Overall vision for project pitched. Various types of systems mooted. Initial concept established
Key outcomes	Chris willing to be primary stakeholder. Project vision established

2) Initial requirement discovery

Attendees	Chris Hann, Brendan Booth, Chanelle Duffy
Date	24 th August
Location	Link Rm 502
Discussion points	More precise design concept outlined. Feedback on feasibility and advice on realistic approach to take. Various requirements discussed.
Key outcomes	Complete stakeholder requirements to be produced and submitted for review.

3) Final stakeholder requirements review

Attendees	Chris Hann, Brendan Booth, Chanelle Duffy, Edward Birchler-Stockdill
Date	29 th August
Location	Link Rm 502
Discussion points	Feedback on formalised stakeholder requirements document. Various corrections, and discussion.
Key outcomes	Stakeholder requirements finalised and to be ready for Proposal

4) Prototype demonstration and evaluation

Attendees	Chris Hann, Brendan Booth, Chanelle Duffy, Edward Birchler-Stockdill
Date	18 th September
Location	Power electronics lab, Rm 104
Discussion points	Demonstration of prototype. Observation and discussion of behaviour and general assessment of prototype. Examination of prototype performance against requirements and clear fulfilment of tangible requirements demonstrated. Expression of enthusiasm for further development.
Key outcomes	Prototype assessed against requirements successfully. Further development sanctioned